

SIH 2026

BLOCKCHAIN / HYPERLEDGER FABRIC — PRODUCT REQUIREMENTS DOCUMENT

Permissioned Trust Layer • Chaincode • Identity • Trusted Events • Audit

Purpose: *This document defines the blockchain component only. It covers the Hyperledger Fabric network, identities, channel, chaincode package, three logical contracts, trusted state transitions, audit/verification events and the Gateway-facing contract. PostgreSQL, Redis, IPFS and FastAPI implementation details are deliberately outside this PRD.*

The current repository TRD specifies Hyperledger Fabric as the permissioned trust layer, with chaincode enforcing critical state transitions and authorization. It defines one chaincode package containing three logical contracts: TraceabilityContract, IncidentContract and AuditContract. [Return to view 1](#)

Core philosophy: Blockchain is a trust component of the food-traceability platform, not the entire platform.
Large/query-heavy data stays off-chain; Fabric stores the critical trusted state/events needed to prove who did what and when.

1. Blockchain Product Vision

The blockchain layer exists because multiple organizations in a food supply chain should be able to participate in a shared history without one organization being able to silently rewrite the trusted record. The platform needs trusted records for product identity, batch lifecycle, custody, transformation, lineage-related state, incidents, recalls and scan/verification audits. The repository PRD describes the core goal as trusted multi-organization event history and specifically requires Fabric + chaincode for critical rules/events. [cite](#) [turn0view0](#)

Fabric therefore acts as the trust anchor. FastAPI is the application orchestrator, PostgreSQL is the operational/read model, Redis accelerates reads, and IPFS stores large evidence. Fabric does not replace those systems.

2. Blockchain Goals

1. Provide a permissioned multi-organization ledger.
2. Give critical product/batch/unit state transitions a trusted transaction history.
3. Enforce critical authorization and lifecycle rules through chaincode.
4. Record trusted custody and transformation events.
5. Record scan and authenticity verification interactions without treating them as custody changes.
6. Record incident, block, recall and accountability/escalation commitments.
7. Provide transaction IDs/events that FastAPI can correlate with PostgreSQL read-model records.
8. Keep on-chain payloads intentionally compact and avoid using Fabric as a large document database.

3. Non-Goals

Not owned by blockchain	Handled by
Fast trace/history queries and relational lineage read models	PostgreSQL
Large certificates/lab reports/feedback attachments	IPFS
Caching	Redis
HTTP API, request validation and orchestration	FastAPI
Consumer/stakeholder UI	Frontend
EPCIS/event streaming, IoT, AI/ML and advanced privacy	Later modules

4. Network Model

The Base Model requires a permissioned Fabric network in which participating organizations have identities and authorized peers/clients. The exact number of physical nodes is an infrastructure/deployment decision; the logical model must support multiple organizations and organization-aware transactions.

Logical flow:

Organization Identity / MSP → Client / Fabric Gateway → Channel → Endorsing Peers → Ordering Service → Committed Ledger

Component	Blockchain requirement
MSP / Identity	Identify organizations and users/clients that submit transactions.
Peers	Maintain ledger/world state and execute chaincode for endorsement.
Ordering service	Order transactions for the channel.
Channel	Provide the shared permissioned ledger context for the Base Model.
Chaincode	Enforce trusted business/state rules.
Fabric Gateway	Application-facing transaction/query entry point used by FastAPI.

The exact organization/node topology is not fully specified in the current PRD/TRD. Therefore Phase 1 should implement the minimum working permissioned topology required for the demonstration, without pretending a production consortium topology has already been finalized.

5. Identity Requirements

1. Every participating organization must have a Fabric-recognized identity domain/MSP.
2. Transactions must carry the submitting actor's identity.

3. Chaincode must be able to determine the actor's organization and relevant role/attributes.
4. FastAPI authorization and Fabric authorization are complementary: FastAPI rejects unauthorized API calls, while chaincode protects critical ledger transitions.
5. Private identity material must not be stored in PostgreSQL, Redis or IPFS.

6. One Chaincode Package, Three Logical Contracts

The TRD explicitly freezes the blockchain design as **one chaincode package containing three logical contracts**. Do not create a separate chaincode package for every feature. [■cite■turn0view1■](#)

Logical Contract	Purpose	Primary trusted data
TraceabilityContract	Product, batch, unit, lifecycle, custody, receipt, transfer, transformation and lineage-related trusted state.	Product/batch/unit assets and lifecycle/custody/transformation events.
IncidentContract	Investigation, incident status, blocking, recall actions and accountability/escalation commitments.	Incident/recall/accountability state.
AuditContract	Scan, verification and audit-only interactions.	Scan/verification audit events.

Contract boundary rule: state/custody change → TraceabilityContract; incident/recall commitment → IncidentContract; interaction/verification only → AuditContract.

7. Ledger Asset Model

Asset	Minimum trusted fields
Product	product_id, product type/name, creator organization, created_at, status/reference metadata.
Batch	batch_id, product_id, current state, current custodian/organization, parent/reference metadata, created_at, updated_at.
Unit	unit_id, batch_id, product_id, state, current custodian/reference, created_at, updated_at.
Incident	incident_id, linked entity/batch/unit, category/status, reporter/reference, timestamps, evidence CIDs where contract requires.
Recall Action	recall_id, affected scope/reference, authority, status, timestamp and transaction context.
Audit Event	event_id, event type, entity, actor/organization, timestamp, verification result/reference.

The exact full field-level chaincode schema should remain intentionally smaller than PostgreSQL. Fabric stores what is required for trust, validation and audit; PostgreSQL stores the richer operational/query model.

8. TraceabilityContract Requirements

Function	Requirement
registerProduct	Create a trusted product identity; reject duplicate identity.
registerBatch	Create a trusted batch linked to a product and authorized organization.
validateBatch	Move a batch through the permitted validation transition only for an authorized actor.
receiveBatch	Record explicit receipt/custody acceptance by the next authorized stakeholder.
transferBatch	Record explicit custody transfer from the authorized current custodian.
createTransformation	Create derived/child relationship information for a processing/transformation operation.
createUnit	Create a traceable unit derived from a batch.

9. Lifecycle State Enforcement

The chaincode must enforce the lifecycle state machine frozen in Phase 0. The backend may perform pre-validation, but chaincode must reject invalid transitions because the ledger is the trust layer.

Example	Allowed behavior
REGISTERED → VALIDATED	Only an authorized validation transaction.
VALIDATED → IN_TRANSIT	Only the relevant business/custody operation.
IN_TRANSIT → RECEIVED	Explicit receive by the intended recipient.
RECEIVED → PROCESSED	Authorized processing/transformation.
PROCESSED → AVAILABLE	Authorized release/availability transition.
Any relevant state → BLOCKED	Only authorized incident/block action.
BLOCKED → RECALLED	Authorized recall action.
The exact transition graph must match the Phase 0 state-machine document; chaincode must not silently invent alternative states.	

10. Custody Principle

A critical feature of this system is that every stakeholder can see the previous permitted history and then explicitly record their own interaction/business action. However, merely scanning a QR code must never transfer custody. Receipt and transfer are separate transactions.

Example: Manufacturer scans → AuditContract records scan. Manufacturer actually receives product → TraceabilityContract.receiveBatch records custody. Manufacturer transfers it → TraceabilityContract.transferBatch records the transfer.

11. Transformation and Lineage

When a processor/manufacture converts a parent batch into a derived batch or units, the trusted transaction must preserve the parent-child relationship/reference required for later lineage reconstruction. PostgreSQL will hold the richer indexed graph; Fabric needs enough trusted relationship information to prove that the transformation was authorized and occurred.

1. Verify actor is authorized to transform.
2. Verify input batch exists and is in a permissible state.
3. Create child/derived identity.
4. Record parent/reference relationship.
5. Record transformation actor and timestamp.
6. Return transaction ID/event reference for backend synchronization.

12. AuditContract Requirements

The AuditContract exists because the platform needs a trusted record of interactions without confusing those interactions with business-state changes. The PRD explicitly states that every scan is an auditable interaction and that a QR scan is not a state-changing event. ■cite■turn0view0■

Function	Requirement
recordScan	Record who/what/when/which credential was scanned and the verification/result context without changing custody.
recordVerification	Record the result of inner credential/authenticity verification without changing ownership/state.

13. Scan Flow

Outer OR:

Scan → FastAPI resolves identity → permitted history returned → AuditContract.recordScan() → audit transaction committed

Inner credential:

Scan inner credential → FastAPI verification → compare/validate trusted reference → AuditContract.recordVerification() → verification result returned

The precise cryptographic construction for the inner credential is intentionally not frozen here. The blockchain requirement is that the verification result/interaction can be trusted and audited; the credential-generation implementation belongs to the later QR/authenticity technical specification.

14. IncidentContract Requirements

Function / capability	Requirement
createIncident	Create trusted incident/complaint reference associated with a product/batch/unit.
updateIncidentStatus	Record authorized investigation/status transitions.
escalateIncident	Record escalation from nearest accountability layer to an upstream layer when rules trigger.
blockBatch / blockUnit	Create trusted blocking state for affected scope.
createRecallAction	Record authorized recall/corrective-action commitment.
closeIncident	Record authorized closure after investigation/corrective action.
Consumer feedback itself is not blindly treated as proof of contamination. The PRD calls for pattern detection, thresholds and verification workflows to mitigate false complaints. ■cite■turn0view0■	

15. Accountability and Escalation on the Ledger

The application determines complaint patterns and accountability logic; Fabric records the critical commitments and resulting incident/escalation/block/recall transitions. This keeps complex aggregation/query work off-chain while ensuring that important decisions cannot be silently rewritten.

Conceptual progression:

One complaint → local accountability record
 Multiple similar complaints → pattern detected off-chain
 Threshold crossed → escalation decision
 Authorized escalation/incident state → IncidentContract record
 Verified systemic issue → block/recall transaction

The exact numerical thresholds are application configuration, not immutable chaincode constants unless the team deliberately chooses otherwise.

16. Recall and Corrective Action

1. An authorized actor submits a block/recall request through FastAPI.
2. FastAPI validates role and affected scope.
3. IncidentContract verifies the actor's ledger authorization and current state.
4. Fabric records the trusted block/recall transition.
5. FastAPI receives the commit event and updates PostgreSQL.
6. Redis trace/risk caches are invalidated.
7. Stakeholders can retrieve the trusted recall reference and affected scope.

17. Evidence and IPFS Relationship

Large evidence files are not stored directly on Fabric. The PRD requires off-chain evidence in IPFS with references on-chain.

■cite■turn0view0■

Evidence object → IPFS → CID → PostgreSQL metadata + relevant Fabric reference

1. Fabric may store the CID/reference and integrity-related metadata.
2. PostgreSQL stores searchable evidence metadata and the CID.
3. FastAPI controls upload/retrieval authorization.
4. Fabric does not store the large certificate/lab report/attachment itself.

18. Trusted Event Minimum Envelope

Field	Purpose
event_id	Unique event reference.
event_type	REGISTER / VALIDATE / RECEIVE / TRANSFER / TRANSFORM / SCAN / VERIFY / INCIDENT / BLOCK / RECALL etc.
entity_type	PRODUCT / BATCH / UNIT / INCIDENT etc.
entity_id	Affected entity.
actor_id / identity	Submitting Fabric identity.
organization_id / MSP	Actor organization.
timestamp	Trusted event time/context.
transaction_id	Fabric transaction reference.
evidence_cid	Optional evidence reference.

19. Authorization Requirements

Fabric authorization is the second line of defense. The backend should reject invalid requests early, but chaincode must independently verify critical authority and state. Never trust a role or organization identifier supplied only in a client request.

Rule	Requirement
Actor identity	Derived from Fabric identity/transaction context.
Organization	Derived from authenticated Fabric identity/MSP.

Rule	Requirement
Current custodian	Must match transfer/receive preconditions.
Lifecycle state	Must allow the requested transition.
Incident authority	Only authorized roles/organizations may block/recall/escalate.
Duplicate identity	Reject duplicate product/batch/unit creation.

20. Query Requirements

Fabric queries should be used for trusted-state verification and transaction context, not as the main application search engine. The backend should normally use PostgreSQL/Redis for rich trace/history queries and use Fabric to validate or retrieve the trusted ledger state where required.

1. Query product/batch/unit trusted state.
2. Query transaction/audit references.
3. Verify whether an entity is blocked/recalled.
4. Verify credential/scan-related trusted audit records.
5. Retrieve trusted incident/recall state.

21. Fabric → PostgreSQL Synchronization

The blockchain layer must produce sufficient transaction/commit information for the backend to update the operational read model. The agreed architecture explicitly avoids uncontrolled dual writes: Fabric commits first, then the backend updates PostgreSQL and invalidates Redis. ■cite■turn0view1■

Fabric transaction → Commit event → Backend synchronization handler → PostgreSQL read model → Redis invalidation

A failed PostgreSQL read-model update must not cause the backend to pretend that an already-committed Fabric transaction did not happen. The transaction ID must be retained for reconciliation.

22. Blockchain Error / Rejection Requirements

Failure	Required outcome
Duplicate product/batch/unit	Chaincode rejects transaction.
Invalid lifecycle transition	Chaincode rejects transaction.
Unauthorized actor	Chaincode rejects transaction.
Wrong current custodian	Transfer/receive rejected.
Missing parent/input batch	Transformation rejected.
Invalid incident/recall state	IncidentContract rejects transaction.
Invalid audit request	AuditContract rejects malformed/unauthorized record.

23. Chaincode Determinism

1. Chaincode functions must be deterministic.
2. Do not rely on local filesystem state.
3. Do not make external network calls from chaincode.
4. Do not generate application secrets inside chaincode.
5. Use transaction context and ledger state for validation.
6. Keep large files outside the ledger.
7. Keep business state compact enough for practical ledger operation.

24. Security Requirements

1. Use Fabric MSP identities for organization/actor identity.
2. Protect private keys and certificates outside application source control.
3. Use separate development identities from production identities.
4. Do not expose Fabric administrative credentials through FastAPI.
5. Chaincode must verify critical permissions independently.
6. Ledger records should avoid unnecessary personal/consumer data.
7. Evidence should be referenced through CIDs rather than placing sensitive files directly on-chain.

25. Testing Requirements

Test level	Must prove
Chaincode unit tests	Assets, validation rules and state transitions.
Authorization tests	Allowed actor succeeds; unauthorized actor fails.
State-machine tests	Invalid transitions are rejected.
Ledger query tests	Created assets and trusted states can be queried.
Audit tests	Scan/verification is recorded without changing custody.
Transformation tests	Parent/child relationships are correctly recorded.
Incident tests	Block/recall/escalation rules reject invalid operations.
Gateway integration	FastAPI can invoke/query chaincode and receive transaction results.
End-to-end	Product → Batch → Validate → Receive/Transfer → Scan works with ledger references.

26. Phase 1 Blockchain Deliverables

1. Runnable Fabric network for the Base Model.
2. Configured MSP identities for the participating demo organizations.
3. Working channel.
4. One chaincode package.
5. Three logical contracts.
6. Product, Batch and Unit asset definitions.
7. registerProduct().
8. registerBatch().

9. validateBatch().
10. State-transition and authorization checks.
11. Gateway-compatible transaction/query interfaces.
12. Chaincode tests.
13. Clear deployment/invoke commands for D3.

27. Implementation Priority

Priority	Blockchain work	Target
P0	Fabric network + MSP + channel + chaincode skeleton	Phase 1
P0	TraceabilityContract: product/batch + register/validate	Phase 1
P0	Gateway transaction/query interface	Phase 1 integration
P1	Receive + transfer	Cycle 2
P1	Transformation + unit + lineage references	Cycle 3
P1	AuditContract scan + verification	Cycle 4
P1	IncidentContract + accountability + block/recall	Cycles 5–6
P2	Advanced privacy/selective disclosure, streaming/EPCIS integrations	Later

28. Blockchain Developer Handoff Contract

The blockchain developer must hand FastAPI developer a small, explicit contract rather than asking them to understand the Fabric internals.

1. How to start the Fabric network.
2. How to deploy/upgrade the chaincode.
3. Gateway connection configuration.
4. Required Fabric identity/client configuration.
5. Transaction names and exact request payloads.
6. Query names and exact response payloads.
7. Expected transaction/commit response.
8. Error/rejection codes or recognizable error messages.
9. How to run chaincode tests.
10. Which events/commit notifications the backend must consume.

29. Definition of Done — Blockchain Base Layer

1. Fabric network runs reproducibly.
2. Required organizations have working identities.
3. One chaincode package contains the three logical contracts.
4. Product/batch creation and validation work through Gateway.
5. Invalid transitions are rejected.
6. Unauthorized actors are rejected.
7. Transaction IDs can be correlated with backend operations.
8. Scan/verification is represented as audit activity, not custody transfer.
9. Large evidence is referenced through CIDs rather than stored on-chain.
10. Blockchain developer can demonstrate the first complete trusted flow without manual ledger manipulation.

30. Final Blockchain Boundary

FastAPI → Fabric Gateway → One Chaincode Package

Inside the package:

TraceabilityContract
→ Product / Batch / Unit / Lifecycle / Custody / Transformation

IncidentContract
→ Incident / Investigation / Accountability / Block / Recall

AuditContract

→ Scan / Verification / Audit-only interactions

*Around it: PostgreSQL holds the rich operational/read model, Redis accelerates reads, and IPFS holds large evidence. Fabric supplies the trusted multi-organization transaction history and critical state validation. This keeps blockchain as a **core trust component** of the solution rather than turning the entire solution into a blockchain application.*

31. Final Principle

Put trust on-chain, put queries off-chain, put heavy evidence in IPFS, and let FastAPI orchestrate the whole experience. That is the blockchain philosophy for this project.

Next natural artifact: the Blockchain TRD / Chaincode Specification — exact asset JSON/state schema, function signatures, transaction payloads, endorsement/authorization rules, Fabric folder structure, Gateway contract, event names and deployment sequence.